

CSE 3204
Formal Language, Automata and Computability

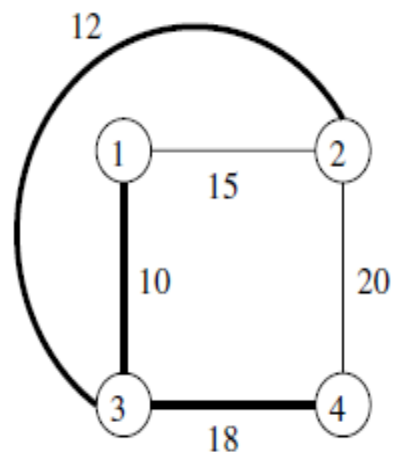


Lecture 24
Intractable Problems

Problems Solvable in Polynomial Time

- A Turing machine **M** is said to be of time complexity **T(n)** if **M** is given an input **w** of length **n**, **M** halts after making at most **T(n)** moves regardless of whether or not **M** accepts.
- interested predominantly in the case where **T(n)** is a polynomial in **n**.
- We say a language **L** is in class **P** if there is some polynomial **T(n)** such that $L = L(M)$ for some deterministic **TM M** of time complexity **T(n)**.

Example: Kruskal's Algorithm



Running time of this algorithm with **m** nodes and **e** edges in a computer is **$O(m+e \log e)$** .

1. Maintain for each node the *connected component* in which the node appears, using whatever edges of the tree have been selected so far. Initially, no edges are selected, so every node is then in a connected component by itself.
2. Consider the lowest-weight edge that has not yet been considered; break ties any way you like. If this edge connects two nodes that are currently in different connected components then:
 - (a) Select that edge for the spanning tree, and
 - (b) Merge the two connected components involved, by changing the component number of all nodes in one of the two components to be the same as the component number of the other.

If, on the other hand, the selected edge connects two nodes of the same component, then this edge does not belong in the spanning tree; it would create a cycle.

3. Continue considering edges until either all edges have been considered, or the number of edges selected for the spanning tree is one less than the number of nodes. Note that in the latter case, all nodes must be in one connected component, and we can stop considering edges.

Example: Kruskal's Algorithm

- A simpler implementation will require:
 - A table gives the current component of each node
 - We pick the lowest weight remaining edge in $O(e)$ time.
 - We find the components of the two nodes connected by the edge in $O(m)$ time
 - Merging nodes in different components requires $O(m)$ time
 - Total running time is $O(e(m+e))$.

Translating into the input of a TM

- with TM, we only think of problems as languages and the only output is yes or no or accept/reject.
- The MWST problem could be think as given this graph G and limit W , does G have a spanning tree of weight W or less?
- In the theory of intractability, we generally want to argue that a problem is hard not easy and the fact that a yes-no version of a problem is hard implies that a more standard version, where a full answer must be computed is also hard.
- While we might think informally of the size of a graph as the number of its nodes or edges, the input to a TM is a string over a finite alphabet.
- Thus nodes and edges must be encoded suitably.

Translating into the input of a TM

- Assign integers 1 through m to the nodes.
- Begin the code with the value of m in binary and the weight limit W in binary separated by a comma.
- If there is an edge between nodes i and j with weight w , place (i, j, w) in the code, where i, j and w are coded in binary.

Thus, one of the possible codes for the graph of Fig. 10.1 with limit $W = 40$ is

100,101000(1,10,1111)(1,11,1010)(10,11,1100)(10,100,10100)(11,100,10010)

Translating into the input of a TM

If we represent inputs to the MWST problem as in Example 10.2, then an input of length n can represent at most $O(n/\log n)$ edges. It is possible that m , the number of nodes, could be exponential in n , if there are very few edges. However, unless the number of edges, e , is at least $m - 1$, the graph cannot be connected and therefore will have no MWST, regardless of its edges. Consequently, if the number of nodes is not at least some fraction of $n/\log n$ there is no need to run Kruskal's algorithm at all; we simply say "no; there is no spanning tree of that weight."

Thus, if we have an upper bound on the running time of Kruskal's algorithm as a function of m and e , such as the upper bound $O(e(m+e))$ developed above, we can conservatively replace both m and e by n and say that the running time, as a function of the input length n is $O(n(n+n))$, or $O(n^2)$. In fact, a better

Translating into the input of a TM

language with useful data structures such as arrays and pointers. However, we claim that in $O(n^2)$ steps we can implement the version of Kruskal's algorithm described above on a multitape TM. The extra tapes are used for several jobs:

1. One tape can be used to store the nodes and their current component numbers. The length of this table is $O(n)$.
2. A tape can be used, as we scan the edges on the input tape, to hold the currently least edge-weight found, among those edges that have not been marked "used." We could use a second track of the input tape to mark those edges that were selected as the edge of least remaining weight in some previous round of the algorithm. Scanning for the lowest-weight, unmarked edge takes $O(n)$ time, since each edge is considered only once, and comparisons of weight can be done by a linear, right-to-left scan of the binary numbers.
3. When an edge is selected in a round, place its two nodes on a tape. Search the table of nodes and components to find the components of these two nodes. This task takes $O(n)$ time.

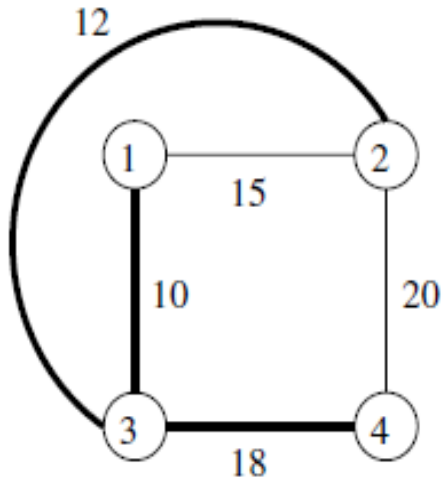
4. A tape can be used to hold the two components, i and j , being merged when an edge is found to connect two previously unconnected components. We then scan the table of nodes and components, and each node found to be in component i has its component number changed to j . This scan also takes $O(n)$ time.

s steps, a single-tape TM can do in $O(s^2)$ steps. Thus, if the multitape TM takes $O(n^2)$ steps, then we can construct a single-tape TM to do the same thing in $O((n^2)^2) = O(n^4)$ steps. Our conclusion is that the yes-no version of the MWST problem, "does graph G have a MWST of total weight W or less," is in $\mathcal{P}$.

Nondeterministic Polynomial Time

- Problems that can be solved by a nondeterministic TM that runs in polynomial time.
- A language L is in the class NP if there is a NTM M and a polynomial time complexity $T(n)$ such that $L = L(M)$ when M is given an input of length n there are no sequences of more than $T(n)$ moves of M .
- since every DTM is a NTM, $P \subseteq NP$.
- a NTM running in polynomial time has the ability to guess an exponential number of possible solutions to a problem and check each one in polynomial time, in parallel.
- Open question whether $P=NP$.

Traveling Salesman Problem



- whether the graph has a Hamilton circuit of total weight at most W
- A Hamilton circuit is a set of edges that connect the nodes into a single cycle, with each node appearing exactly once.
- The number of edges on a Hamilton circuit must equal the number of nodes in the graph.
- Hamilton cycle is (1, 2, 4, 3, 1) of weight 63.
- In a m -node graphs, the number of distinct cycles grows as $O(m!)$ more than 2^{cm} for any constant c .

It appears that all ways to solve the TSP involve trying essentially all cycles and computing their total weight. By being clever, we can eliminate some obviously bad choices. But it seems that no matter what we do, we must examine an exponential number of cycles before we can conclude that there is none with the desired weight limit W , or to find one if we are unlucky in the order in which we consider the cycles.

On the other hand, if we had a nondeterministic computer, we could guess a permutation of the nodes, and compute the total weight for the cycle of nodes in that order. If there were a real computer that was nondeterministic, no branch would use more than $O(n)$ steps if the input was of length n . On a multitape NTM, we can guess a permutation in $O(n^2)$ steps and check its total weight in a similar amount of time. Thus, a single-tape NTM can solve the TSP in $O(n^4)$ time at most. We conclude that the TSP is in $\mathcal{NP}$.

Polynomial –Time Reduction

Our principal methodology for proving that a problem P_2 cannot be solved in polynomial time (i.e., P_2 is not in $\mathcal{P}$) is the reduction of a problem P_1 , which is known not to be in $\mathcal{P}$, to P_2 .² The approach was suggested in Fig. 8.7, which we reproduce here as Fig. 10.2.

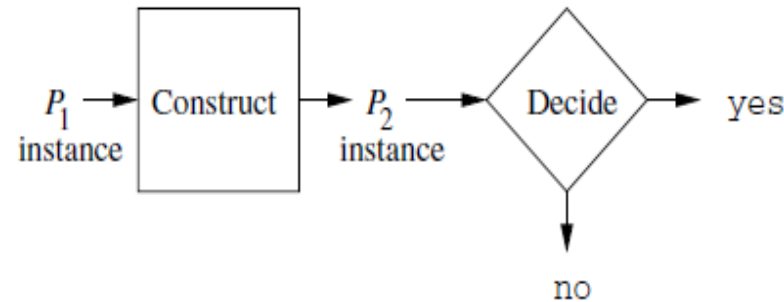


Figure 10.2: Reprise of the picture of a reduction

Suppose we want to prove the statement “if P_2 is in $\mathcal{P}$, then so is P_1 .” Since we claim that P_1 is *not* in $\mathcal{P}$, we could then claim that P_2 is not in $\mathcal{P}$ either. However, the mere existence of the algorithm labeled “Construct” in Fig. 10.2 is not sufficient to prove the desired statement.

For instance, suppose that when given an instance of P_1 of length m , the algorithm produced an output string of length 2^m , which it fed to the hypothetical polynomial-time algorithm for P_2 . If that decision algorithm ran in, say, time $O(n^k)$, then on an input of length 2^m it would run in time $O(2^{km})$, which is exponential in m . Thus, the decision algorithm for P_1 takes, when given an input of length m , time that is exponential in m . These facts are entirely consistent with the situation where P_2 is in $\mathcal{P}$ and P_1 is not in $\mathcal{P}$.

Polynomial –Time Reduction

Even if the algorithm that constructs a P_2 instance from a P_1 instance always produces an instance that is polynomial in the size of its input, we can fail to reach our desired conclusion. For instance, suppose that the instance of P_2 constructed is of the same size, m , as the P_1 instance, but the construction algorithm itself takes time that is exponential in m , say $O(2^m)$. Now, a decision algorithm for P_2 that takes polynomial time $O(n^k)$ on input of length n only implies that there is a decision algorithm for P_1 that takes time $O(2^m + m^k)$ on input of length m . This running time bound takes into account the fact that we have to perform the translation to P_2 as well as solve the resulting P_2 instance. Again it would be possible for P_2 to be in $\mathcal{P}$ and P_1 not.

The correct restriction to place on the translation from P_1 to P_2 is that it requires time that is polynomial in the length of its input. Note that if the translation takes time $O(m^j)$ on input of length m , then the output instance of P_2 cannot be longer than the number of steps taken, i.e., it is at most cm^j for some constant c . Now, we can prove that if P_2 is in $\mathcal{P}$, then so is P_1 .

NP-Complete Problem

for being in $\mathcal{NP}$ but not in $\mathcal{P}$. Let L be a language (problem). We say L is *NP-complete* if the following statements are true about L :

1. L is in $\mathcal{NP}$.
2. For every language L' in $\mathcal{NP}$ there is a polynomial-time reduction of L' to L .

man Problem, which we introduced in Section 10.1.4. Since it appears that $\mathcal{P} \neq \mathcal{NP}$, and in particular, all the NP-complete problems are in $\mathcal{NP} - \mathcal{P}$, we generally view a proof of NP-completeness for a problem as a proof that the problem is not in $\mathcal{P}$.

Theorem 10.4: If P_1 is NP-complete, P_2 is in $\mathcal{NP}$, and there is a polynomial-time reduction of P_1 to P_2 , then P_2 is NP-complete.

PROOF: We need to show that every language L in $\mathcal{NP}$ polynomial-time reduces to P_2 . We know that there is a polynomial-time reduction of L to P_1 ; this reduction takes some polynomial time $p(n)$. Thus, a string w in L of length n is converted to a string x in P_1 of length at most $p(n)$.

We also know that there is a polynomial-time reduction of P_1 to P_2 ; let this reduction take polynomial time $q(m)$. Then this reduction transforms x to some string y in P_2 , taking time at most $q(p(n))$. Thus, the transformation of w to y takes time at most $p(n) + q(p(n))$, which is a polynomial. We conclude that L is polynomial-time reducible to P_2 . Since L could be any language in $\mathcal{NP}$, we have shown that all of $\mathcal{NP}$ polynomial-time reduces to P_2 ; i.e., P_2 is NP-complete. $\square$

NP-Complete Problem

Theorem 10.5 : If some NP-complete problem P is in $\mathcal{P}$, then $\mathcal{P} = \mathcal{NP}$.

PROOF: Suppose P is both NP-complete and in $\mathcal{P}$. Then all languages L in $\mathcal{NP}$ reduce in polynomial-time to P . If P is in $\mathcal{P}$, then L is in $\mathcal{P}$, as we discussed in Section 10.1.5. $\square$

- Section 10.1 of Chapter 10.